AI Machine Health Monitor

B09 – FAIR-AI-09 | Complete Project Guide

Domain: Predictive Maintenance | Industrial AI

Project objective: Detect unusual machine behavior from sensor data using data analysis, machine learning, and a practical dashboard.

1. What Is the Project About?

The AI Machine Health Monitor analyzes machine sensor readings and identifies unusual operating behavior. Sensors may measure temperature, vibration, pressure, rotational speed, voltage, or other machine conditions.

The system will:

• Collect or load sensor data.

• Learn normal machine behavior.

• Detect unusual sensor patterns.

• Display trends and alerts on a dashboard.

Important: The project mainly focuses on anomaly detection. It does not necessarily predict the exact date on which a machine will fail.

2. What You Have to Develop

You need to create a working software application using a public machine-sensor dataset.

Proposed workflow:

Machine Sensor Dataset → Data Loading → Data Cleaning → Visualization → Machine Learning Model → Anomaly Detection → Dashboard and Alerts

Illustrative example:

Temperature	Vibration	Status
45°C	0.20	Normal
47°C	0.22	Normal
49°C	0.25	Normal
85°C	1.20	Anomaly
90°C	1.50	Anomaly

The values above are only illustrative and are not real machine safety limits.

3. Hardware Requirements

Option 1: Software-Only Project (Recommended for Beginners)

Physical hardware is not required. You can complete the project using a laptop or desktop and a public machine-sensor dataset.

Hardware	Purpose
Laptop/Desktop	Development and execution

Minimum 8 GB RAM	Comfortable development
Internet connection	Download datasets and libraries
Optional GPU	Not required for the initial project

Option 2: Hardware-Based IoT Prototype

For a real-time demonstration, you can monitor a small low-voltage DC motor using an ESP32 and sensors.

Hardware	Purpose
ESP32	Microcontroller with Wi-Fi
MPU6050	Vibration and motion measurements
DS18B20	Temperature measurement
OLED display	Display sensor readings
Jumper wires	Electrical connections
Breadboard	Circuit assembly
Small DC motor	Demonstration machine

Hardware workflow: Sensors → ESP32 → Wi-Fi → Python Backend → AI Anomaly Detection → Dashboard

Use a low-voltage demonstration setup. Do not connect experimental hardware to industrial machinery without proper safety supervision.

4. Technologies to Use

Component	Recommended Technology	Purpose
Programming language	Python	Main development language
Data processing	Pandas, NumPy	Load, clean, and prepare data
Visualization	Matplotlib / Plotly	Create sensor trend graphs
Machine learning	Scikit-learn	Anomaly detection
ML model	Isolation Forest	Detect unusual observations
Dashboard	Streamlit	Build a web dashboard
Dataset	NASA C-MAPSS / public dataset	Machine sensor data
Development	VS Code / Jupyter Notebook	Coding and testing
Database	SQLite (optional)	Store sensor and alert history
IoT hardware	ESP32 (optional)	Collect real-time readings

5. Example Python Code

Load sensor data using Pandas:

```
import pandas as pd

data = pd.read_csv("sensor_data.csv")
```

```
print(data.head())
print(data.describe())
```

Basic Isolation Forest implementation:

```
from sklearn.ensemble import IsolationForest

model = IsolationForest(
contamination=0.05,
random_state=42
)

model.fit(X_train)
predictions = model.predict(X_test)
```

In this example, a prediction of 1 indicates a normal observation and -1 indicates an anomaly. Features, training data, and contamination must be selected and evaluated appropriately.

6. Project Ideas

Idea 1: AI Machine Health Monitor – Software Only

• Upload a CSV dataset.

• Clean and analyze sensor data.

• Display sensor graphs.

• Train an Isolation Forest model.

• Detect anomalies.

• Show alerts.

• Export anomaly results.

Difficulty: Beginner to intermediate. This is the most direct approach for the given project brief.

Idea 2: Real-Time Motor Health Monitor – Hardware + AI

• Measure temperature and vibration in real time.

• Send readings through ESP32 Wi-Fi.

• Display live readings on a dashboard.

• Detect unusual behavior.

• Store sensor history.

Difficulty: Intermediate. It requires both electronics and software integration.

Idea 3: Advanced Machine Health Prediction System

• Analyze sensor degradation.

• Detect anomalies.

• Visualize machine degradation.

• Estimate remaining useful life (RUL).

• Compare historical machine behavior.

Difficulty: Intermediate to advanced. This extends beyond the basic anomaly-detection requirement.

7. Using the NASA C-MAPSS Dataset

NASA C-MAPSS is a dataset used to study turbofan engine degradation through operating conditions and sensor measurements.

Possible activities:

• Load engine sensor data.

• Explore how readings change over time.

• Select relevant features.

• Develop an anomaly-detection approach.

• Visualize engine behavior.

• Evaluate whether the model identifies unusual observations.

C-MAPSS includes run-to-failure degradation data. Therefore, define your training period, testing period, and evaluation method carefully.

8. Five-Week Development Plan

Week	Tasks	Expected Output
Week 1	Learn Python, Pandas, NumPy, basic statistics, and Matplotlib.	Dataset loaded and understood
Week 2	Download data, clean data, handle missing values, and create graphs.	Clean dataset and visualizations
Week 3	Learn anomaly detection, implement Isolation Forest, and analyze predictions.	Working ML model
Week 4	Create Streamlit dashboard, graphs, CSV upload, and alerts.	Working web dashboard
Week 5	Test, evaluate false positives/negatives, improve UI, and prepare documentation.	Prototype, report, and presentation

This is an example schedule. Adjust it according to your college deadline and experience.

9. Extra Features

• **Machine health status:** Show Normal, Warning, or Critical categories based on model output and defined thresholds.

• **Anomaly history:** Display the time, sensor, and status of previous alerts.

• **Sensor trend graphs:** Show temperature, vibration, and other sensor readings over time.

• **CSV upload:** Allow users to upload their own machine dataset.

• **Email notifications:** Send an email when an anomaly is detected, using an email service or SMTP configuration.

10. Final Project Demonstration Checklist

• Machine-sensor dataset.

• Data cleaning using Pandas.

• Sensor visualization.

• Machine-learning-based anomaly detection.

• Working Streamlit dashboard.

• Anomaly results and interpretation.

• Project report and presentation.

• Optional hardware demonstration.

The main objective is to create a working and understandable system rather than simply using many technologies.

11. Recommended Final Project Design

Suggested title: AI-Based Machine Health Monitoring and Anomaly Detection Using Machine Learning

Technology stack: Python, Pandas, NumPy, Scikit-learn, Isolation Forest, Streamlit, and NASA C-MAPSS or another suitable public sensor dataset.

Hardware: Not required initially. ESP32 and sensors can be added later for a physical prototype.

Final output: A web dashboard that loads machine sensor data, analyzes it, detects unusual behavior, and displays the results.

12. Suggested Starting Point

Start with the software-only version. First build the dataset analysis and anomaly detection model, then create the Streamlit dashboard. Once the software works, consider adding ESP32 hardware for real-time data collection.